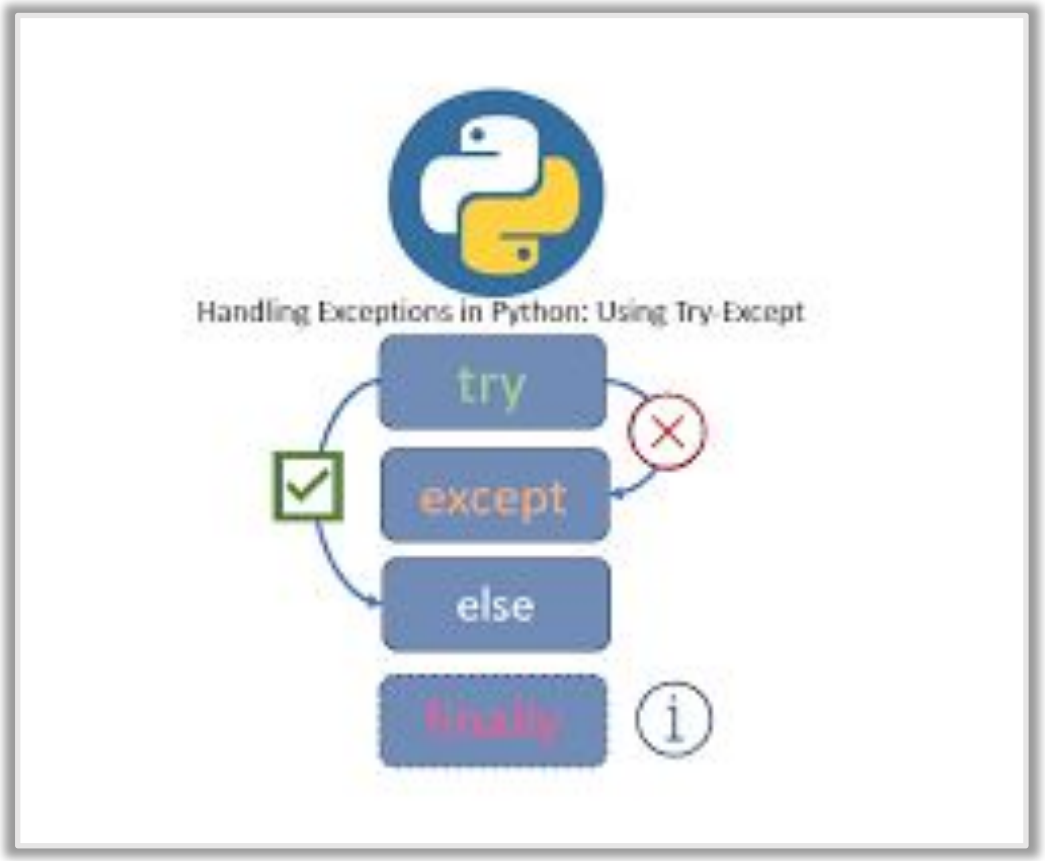
Exception Handling in Python

Exception Handling

- In Python, **Exception Handling** is the process of responding to unwanted or unexpected events when a computer program runs.
- Instead of the program "crashing" or stopping abruptly, exception handling allows the code to catch the error, deal with it gracefully, and continue executing.
- Think of it like a safety net for your code. If something goes wrong—like trying to divide by zero or opening a file that doesn't exist—the "exception" is caught before it breaks everything.



Error Vs Exception

What is an Error?

- An **Error** is a **serious problem** that occurs in a program and **cannot be handled** using normal exception handling.
- Errors usually stop the program immediately.

- **Characteristics of Errors**
  - Occur due to **syntax mistakes** or **system-level issues**
  - Cannot be recovered easily
  - Program **terminates abruptly**
  - Mostly occur **before execution** (compile time)

What is an Exception?

- An **Exception** is a **runtime problem** that occurs **during execution** and **can be handled** using try-except.
- Exceptions allow the program to continue safely.

- **Characteristics of Exceptions**
  - Occur at **runtime**
  - Can be handled and managed
  - Prevent program crash
  - Mostly caused by **logical mistakes or invalid input**

Error Vs Exception

Feature	Error	Exception
Definition	Serious issue in program	Runtime issue
Handling	Cannot be handled	Can be handled
Occurs at	Compile time / system level	Runtime
Program flow	Stops execution	Can continue
Cause	Syntax, memory, system failure	Invalid input, logic issue
Examples	SyntaxError, IndentationError	ValueError, ZeroDivisionError

The Core Structure of Exception Handling

1. **try:** This is where you put the code you want to run, but think might cause a problem. Python will attempt to run every line in this block.
2. **except:** This block is skipped if everything goes well. However, if an error happens inside the try block, Python immediately jumps here to handle it.

3. The else block: "Only on Success“

The else block runs only if no exceptions were raised in the try block. It’s the perfect place for code that should only execute if the previous operation was 100% successful.

4. The finally block: "No Matter What“

The finally block is the ultimate safety net. It runs regardless of whether an error happened or not. Even if the program crashes or uses a return statement, Python ensures the finally block executes before moving on.

Common Error Types in Python ⚠️

Error Type	Cause
SyntaxError	Invalid syntax
IndentationError	Wrong indentation
NameError	Variable not defined
TypeError	Wrong data type
ValueError	Invalid value
ZeroDivisionError	Divide by zero
IndexError	Invalid index
KeyError	Missing dictionary key
AttributeError	Invalid attribute
ImportError	Module not found
FileNotFoundError	File missing

try Block with Multiple except Block

- In Python, a single block of code may produce **different types of exceptions** at runtime. To handle each possible exception **individually**, Python allows **multiple except blocks** with a single try block.

  This ensures:

  - Precise error handling
  - Better debugging
  - Smooth program execution

Why Multiple except Blocks Are Needed

- A single except block is **not sufficient** when:
- Different errors require **different handling logic**
- User input may cause **multiple runtime issues**
- Program interacts with files, input, arithmetic, or data types

Example Situations:

- ValueError → invalid user input
- ZeroDivisionError → divide by zero
- FileNotFoundError → missing file

try Block with Multiple except Block

Execution Flow

1. Python executes the **try block**
2. If **no exception occurs**, all except blocks are skipped
3. If an exception occurs:
   - Python checks except blocks **from top to bottom**
   - The **first matching except** is executed
4. Remaining except blocks are **ignored**
5. Program continues after the try-except structure

Order of except Blocks (Very Important ⚠️)

```
try:
    print(10 / 0)
except Exception:
    print("General exception")
except ZeroDivisionError:
    print("Division error")
```

ZeroDivisionError is **never reached** because Exception catches all exceptions.

⚠️**Common Mistakes**

- Using except: instead of specific exception types
- Writing general exceptions before specific ones
- Not handling possible runtime errors
- Catching too many exceptions unnecessarily
- Ignoring exception messages

raise in Exception Handling

- The **raise keyword** in Python is used to **manually trigger an exception**. It allows you to stop normal execution and signal that an error condition has occurred.
- It allows developers to **control error conditions**, enforce rules, and create **custom exceptions**.
- Instead of waiting for Python to throw an error automatically (like ZeroDivisionError), you can explicitly raise one yourself.

What is raise?

The raise keyword is used to:

- Trigger built-in exceptions
- Trigger custom exceptions
- Re-raise a caught exception
- Enforce validation rules
- Stop execution when a condition fails
- Unlike runtime errors, raise gives **full control** to the programmer

raise in Exception Handling

Why Use raise?

- Validate business logic
- Enforce constraints
- Handle invalid states
- Create meaningful error messages
- Define custom exceptions

Syntax of raise

```
raise ExceptionType("error message")
```

Common Use Cases

- Input validation
- Business rule enforcement
- API validation
- Financial & banking systems
- Preventing invalid state transitions

Flow of raise

- Condition checked
- raise triggers exception
- Program jumps to nearest except
- If not handled → program terminates

raise in Exception Handling

⚠️ Important Rules

- **raise stops execution immediately -**Code after raise will not run.
- Must raise an Exception object or class
  - raise "error" # invalid
  - raise ValueError("bad")
  - raise ValueError

Built-in Exceptions Commonly Used with raise

Exception	When to use
ValueError	Invalid value
TypeError	Wrong type
ZeroDivisionError	Divide by zero
FileNotFoundError	Missing file
PermissionError	Access denied
RuntimeError	General runtime issue

Assertion in Exception Handling

- **Assertion** in Python is a debugging aid that tests whether a condition is true.
- If the condition is **false**, Python raises an **AssertionError** automatically.
- It is mainly used to **detect programmer mistakes early** and ensure assumptions in the code are valid.

What is an Assertion?

- An assertion is a statement that says:

  "This condition must be true at this point in the program."
- If it's not true → stop execution and raise an error.
- **Basic Syntax**

  assert condition
- Important — Assertions Can Be Disabled ⚠️

  python -O script.py
- **All assert statements are ignored**
  They do NOT execute
  No AssertionError will be raised

Difference b/w Raise and Assertion

Aspect	raise	assert
Purpose	Manually trigger an exception	Verify a condition during debugging
Primary Use	Runtime validation and error handling	Developer assumption checks
Syntax	raise ExceptionType("msg")	assert condition, "msg"
Condition Required	No	Yes (must evaluate True)
Exception Type	Any built-in or custom exception	Always AssertionError
Custom Exception Support	Yes	No
Custom Message	Yes	Yes
Execution	Always executes	Executes only when assertions enabled
Behavior in python -O	Still runs	Completely removed
Production Safety	Safe	Not safe for critical checks
User Input Validation	Recommended	Not recommended
Business Logic Checks	Recommended	Not recommended
Debugging Use	Possible but not primary	Main purpose
Optimization Impact	No change	Skipped entirely
Control Level	Full control over error type	No control over error type